Private Higher School of Engineering & Technology

Al Baraka Bank Tunisia · 44 Branches Nationwide

INTERNSHIP REPORT · 2025 – 2026

A Governed Intelligence for Islamic Banking

Design, Architecture, and Evaluation of the Integrated RAGLab Intelligence Engine & Bank Assistant Platform

"Every answer cites its verbatim source. Every refusal states what is missing. Every architectural decision is measured and auditable."

339	+20 pp	100%	605
Restructured Arabic legal fragments	Structural chunking hit@1 gain (51→71%)	Rank-1 recall on 198 Law 2016-48 units	Automated unit & offline checks green

Prepared by	**Farah AYARI** — Software Engineering Department, ESPRIT
Host Organization	**Al Baraka Bank Tunisia** — Head Office, Centre Urbain Nord Avenue Cheikh Mohamed Fadhel Ben Achour, 1082 Tunis, Tunisia
Company Supervisor	**Mrs. Ahlem Ben Haddoud** — Advisor, Al Baraka Bank Tunisia
Academic Institution	École Supérieure Privée d'Ingénierie et de Technologie (**ESPRIT**)
Academic Year	2025 – 2026

Confidentiality & Reproducibility Note: No customer identifiers, repository URLs, or secret keys appear in this report by design. All quantitative results are directly traceable to frozen benchmark artifacts in the two project repositories.

PRELIMINARY NOTE

Dedication

> *To my family, whose unwavering patience, encouragement, and support made every step of this engineering journey possible;*
>
> *To **Mrs. Ahlem Ben Haddoud**, whose guidance, intellectual rigor, and trust shaped every architectural and product decision in this work;*
>
> *And to everyone who believes that software intelligence, when disciplined and governed, can serve institutional trust.*
>
> **Farah AYARI — ESPRIT**

Acknowledgments

I express my deepest gratitude to **Mrs. Ahlem Ben Haddoud**, my internship supervisor at **Al Baraka Bank Tunisia — Centre Urbain Nord**, for her constant availability, precise operational feedback, and decisive guidance. At every critical juncture — from curating the authoritative regulatory corpus and validating the Arabic linguistic repairs to governing the final switch toward a separated two-system architecture — her judgment turned an experimental technical capability into a responsible banking platform.

I warmheartedly thank the teams at **Al Baraka Bank Tunisia** for welcoming me into the bank's headquarters at the Centre Urbain Nord, entrusting real normative and training documents to this work, and providing candid operational testing that surfaced subtle runtime failure modes invisible in isolated unit tests.

Finally, I thank the faculty and engineering staff of **ESPRIT (École Supérieure Privée d'Ingénierie et de Technologie)** for cultivating rigorous engineering habits — above all, the conviction that every architectural claim must be backed by reproducible measurement rather than mere demonstration.

> **Editorial & Confidentiality Note**
>
> This report is organized around architectural principles, measurable decisions, and real engineering problems solved rather than chronological diary entries. By design, no customer account identifiers, private repository URLs, or secret tokens appear anywhere in this document.

EXECUTIVE SUMMARY

Abstract

In regulated banking, an inaccurate statement regarding a profit rate, fee schedule, statutory deadline, or compliance condition is not a benign conversational slip — it is a direct legal and reputational exposure. This internship designed, implemented, and evaluated a complete, governed software answer to that challenge: **an integrated banking platform in which a bank-operated web application consumes a dedicated Retrieval-Augmented Generation (RAG) intelligence microservice**, both engineered so the system can **never invent figures or unverified rules**.

The platform unites two cooperating systems separated by a strict, versioned HTTP contract:

- **RAGLab (The Intelligence Engine):** A stateless Python/FastAPI grounding engine indexing four unabridged Arabic normative banking documents (Tunisian Banking Law No. 2016-48, Central Bank Circular BCT No. 2019-08, an Internal Islamic Banking Operations Guide, and an Islamic Finance Primer) across 339 structure-aware fragments and 198 declarative legal units. Every generated claim must carry a contiguous verbatim quote whose digits literally appear in the cited passage; when evidence is missing or ambiguous, the engine refuses honestly before or after generation and states what is missing.
- **Bank Assistant (The Digital Bank Edge):** A trilingual (French, Arabic, English with first-class RTL layout) Angular 21, Spring Boot 4 (Java 21 virtual threads), and PostgreSQL 16 web application. As the sole owner of the 15-migration Flyway relational schema, it enforces authentication, token-family rotation, dynamic 12-capability RBAC, two-phase quota metering, pre-call conversation persistence, masked customer slice projection, and a five-tab staff governance console.

Measured on frozen benchmark suites, structure-aware chunking raises `hit@1` by **+20 percentage points** (`51% → 71%`), the declarative Law 2016-48 unit index achieves **100% rank-1 recall** (`100/100/100`), deterministic citation gates reject **100% of adversarial fabrication/injection probes**, and **605 automated verification checks** run green in continuous integration.

Keywords: Retrieval-Augmented Generation (RAG) · Islamic Banking · Citation Verification · Arabic Legal NLP · Spring Boot 4 · Angular 21 · AI Governance · Defense in Depth

Résumé Exécutif (Français)

Dans un environnement bancaire réglementé, une réponse erronée portant sur un taux de marge, un délai légal ou une règle de conformité charaïque ne constitue pas une simple imprécision technique, mais un risque juridique et opérationnel majeur. Ce stage a permis de concevoir, réaliser et évaluer une plateforme intégrée associant **une application web bancaire trilingue (Bank Assistant — Angular 21, Spring Boot 4, Java 21, PostgreSQL 16)** et **un moteur d'intelligence documentaire gouverné (RAGLab — Python 3.11, FastAPI, ChromaDB 2048-d, BM25)** sur quatre textes réglementaires et métiers arabes de référence (Loi n° 2016-48, Circulaire BCT n° 2019-08, Guide interne des opérations islamiques et manuel **Madkhal**).

L'architecture repose sur une séparation stricte des responsabilités : la passerelle Java détient l'unique schéma relationnel (15 migrations Flyway), l'authentification JWT à rotation de familles, les quotas à verrouillage strié compatible avec les **virtual threads** et l'audit d'administration, tandis que RAGLab garantit la restructuration linguistique des PDF arabes scannés (+20 points de `hit@1`), la recherche dense multilingue, l'index déclaratif des 198 articles de loi (`100 %` au rang 1) et un triple verrou déterministe de citation interdisant tout chiffre ou énoncé non sourcé mot pour mot.

NAVIGATION

Table of Contents

1 Host Organization: Al Baraka Bank Tunisia . . . 1
1.1 Identity, Heritage, and Institutional Standing . . . 1
1.2 Sharia-Compliant Product Universe & Regulatory Framing . . . 1
1.3 The Internship Environment: Centre Urbain Nord Headquarters . . . 1
2 Project Vision: A Governed Banking Intelligence . . . 2
2.1 The Core Business & Engineering Problem . . . 2
2.2 The North Star: Five Non-Negotiable Properties . . . 2
2.3 What Was Delivered: Two Cooperating Systems, One Product . . . 3
3 System Architecture: The Full Picture . . . 3
3.1 End-to-End Platform Topology . . . 3
3.2 Deliberate Architectural Sobriety: Stack Justification . . . 5
3.3 The Two Governed Operational Lifecycles . . . 5
4 The Intelligence Engine — RAGLab: How It Thinks . . . 6
4.1 The Authoritative Arabic Corpus & Its Scanned-PDF Pathology . . . 6
4.2 Three-Stage Structure-Aware Chunking (`restructure` Mode) . . . 7
4.3 High-Dimensional Embeddings, Dual Indexing & Fingerprint Guards . . . 7
4.4 Retrieval Modes: Why Pure Dense Retrieval Remains Default . . . 8
4.5 Deterministic Question Understanding & Bounded Interrogation . . . 8
4.6 Delivery Layer: Three-Stage Citation Gate, Scrubbing & Declarative Law Units . . . 8
5 The Digital Bank — Bank Assistant Web Application . . . 9
5.1 Three Segregated Channels & Identity Isolation . . . 9
5.2 Core Banking Relational Domain (`Flyway V1 .. V15`) . . . 10
5.3 Token-Family Security, Dynamic RBAC & Virtual-Thread Metering . . . 10
5.4 The Staff Control Room & Five-Tab RAGLab Console . . . 11
6 Engineering Challenges Solved: What Broke & How It Was Fixed . . . 11
6.1 Linguistic & Retrieval Breakthroughs . . . 11
6.2 Distributed Systems & Production Runtime Fixes . . . 12
7 Security, Privacy & Data Governance . . . 13
7.1 Seven-Layer Defense-in-Depth Architecture . . . 13
7.2 Personal Data Inventory, Masking & Honest Gap Disclosure . . . 13
7.3 Threat Pass (`THREAT_PASS_6.2`) & End-to-End Traceability . . . 14
8 Deployment, Promotion & Operations . . . 14
8.1 Build Once, Promote Everywhere (`dev → staging → prod`) . . . 14
8.2 Zero-Surprise Rollout & Observability Runbook . . . 15
9 Evaluation Results, Benchmarks & Limitations . . . 15
9.1 Frozen Evaluation Protocol . . . 15
9.2 Consolidated Benchmark & Robustness Scorecard . . . 16
9.3 Explicit Engineering Limitations & Readiness Verdict . . . 16
10 Conclusion & Perspectives . . . 17
10.1 Synthesis of Contributions . . . 17
10.2 Three Engineering Lessons That Outlive the Code . . . 17
10.3 Perspectives & Roadmap Toward Production . . . 17
References & Appendices . . . 18
Primary Repository & Regulatory References . . . 18
Appendix A — Bilingual Technical & Islamic Banking Glossary . . . 18
Appendix B — Consolidated Quantitative Fact Sheet . . . 19
Appendix C — Local Verification & Reproduction Guide . . . 19

List of Engineering Figures

Figure 1 End-to-end platform architecture across Public DMZ, Bank Application Edge, and Private Intelligence Engine. . . . 4
Figure 2 Six-step customer chat execution path combining striped virtual-thread locks, pre-call persistence, and citation gating. . . . 6
Figure 3 RAGLab dual-track architecture: offline linguistic restructuring & indexing (Track A) and online deterministic gating (Track B). . . . 7
Figure 4 Functional architecture of Bank Assistant illustrating strict separation across Public, Client Portal, and Staff Governance surfaces. . . . 9
Figure 5 Comparative hit@1 / hit@3 / hit@5 progression across chunking, fusion, reranking, and declarative unit indexing. . . . 12
Figure 6 Seven-layer defense-in-depth model spanning network isolation, cryptography, RBAC, prompt isolation, PII scrubbing, and audit. . . . 13
Figure 7 Immutable Git-SHA container promotion workflow and production runtime network/backup topology. . . . 15

CHAPTER 1

Host Organization: Al Baraka Bank Tunisia

1.1 Identity, Heritage, and Institutional Standing

Al Baraka Bank Tunisia stands as the first Islamic financial institution established in Tunisia and across the Maghreb region. Founded on **15 June 1983** under the name **Beit Ettamouil Tounsi Saoudi (BEST Bank)** through a joint initiative of the Tunisian State and Sheikh Saleh Abdallah Kamel — founder of the Dallah Al Baraka Group — the institution adopted the unified **Al Baraka Bank Tunisia** identity in 2009. A decisive strategic milestone followed in **2013**, when the bank obtained regulatory approval to transition from an offshore non-resident institution into a **full-service universal resident bank**, enabling nationwide service across retail individuals, liberal professionals, SMEs, and corporate enterprises strictly in accordance with Islamic Sharia principles.

As a flagship subsidiary of **Al Baraka Group B.S.C.** (headquartered in Bahrain Bay and operating more than 600 branches across 12 countries including Bahrain, Jordan, Egypt, Tunisia, Algeria, Turkey, South Africa, Sudan, and Pakistan), Al Baraka Bank Tunisia combines international participative banking standards with deep local economic integration. Today, the bank operates **44 branches** nationwide from its flagship headquarters at the **Centre Urbain Nord** in Tunis. Its sustained operational excellence has been recognized repeatedly, earning **Best Islamic Bank in Tunisia** (2018, 2019, 2021) and an unprecedented **five distinctions at the Islamic Finance News (IFN) Awards 2024**.

1.2 Sharia-Compliant Product Universe & Regulatory Framing

Every financial instrument offered by the bank operates within a rigorous dual governance framework: Sharia supervisory compliance and statutory Tunisian banking law. Specifically, **Tunisian Law No. 2016-48 of 11 July 2016** relating to banks and financial institutions formally codifies participative banking contracts in Articles 12 through 16 and empowers the **Central Bank of Tunisia (BCT)** to regulate their prudential and operational execution via **Circular BCT No. 2019-08**. Remarkably, these exact two statutory texts — alongside the bank's own operational manual and training primer — form the authoritative knowledge base indexed by our platform.

Product Family	Representative Sharia-Compliant Instruments & Scope
Trading & Participative Investment	**Murabaha** (disclosed cost-plus asset financing), **Salam** (deferred-delivery agricultural/commodity sale), **Istisna'a** (progressive manufacturing/construction contract), **Mudaraba** & **Musharaka** (profit-and-loss sharing partnerships), **Sukuk**, and participative investment deposits.
Leasing, Real Estate & Retail Finance	**Ijara** (leasing with optional transfer of ownership), **Beit Al Baraka** (residential property acquisition), **Tas'heelat** (consumer equipment & personal financing), fleet/equipment leasing, and corporate working-capital lines.
International Trade & Daily Banking	Documentary letters of credit (**Crédits documentaires**), international SWIFT remittances, guarantees (**Kafala**), multi-currency accounts, masked bank cards, and digital banking services.

Table 1 — Core Sharia-compliant product families operated by Al Baraka Bank Tunisia.

1.3 The Internship Environment: Centre Urbain Nord Headquarters

Host Site — Al Baraka Bank Tunisia Head Office

Address: Avenue Cheikh Mohamed Fadhel Ben Achour, Centre Urbain Nord, 1082 Tunis, Tunisia
Switchboard: (+216) 71 183 203 · **Headquarters:** (+216) 71 18 65 00 · **SWIFT BIC:** `BEITTNTT`
Supervision: Conducted under the direct guidance of **Mrs. Ahlem Ben Haddoud** (Advisor, Al Baraka Bank Tunisia).

Working directly within the **Centre Urbain Nord** headquarters shaped the engineering ethos of this internship. Located where product structuring, legal/Sharia compliance, customer advisory teams, and information systems

converge daily, the environment made one truth immediately tangible: **in banking software, fluency without verifiability is a liability, whereas honest refusal backed by traceable proof is an operational asset**.

CHAPTER 2

Project Vision: A Governed Banking Intelligence

2.1 The Core Business & Engineering Problem

Modern Islamic banks maintain dense, interlocking normative texts: parliamentary statutes, Central Bank circulars, Sharia board resolutions, internal operational guides, and continuously updated rate/fee schedules. Searching this corpus manually slows frontline staff and leaves prospective customers without timely answers. Yet delegating banking Q&A directly to an unguarded Large Language Model (LLM) introduces severe hazards: standard generative models optimize for plausibility rather than truth, silently blending background pretraining guesses with retrieved snippets and fabricating numbers when passages are incomplete.

Three structural tensions defined our problem space from day one:

1. **The Lexical & Conceptual Asymmetry:** Customers ask practical, everyday questions (**"Can the bank finance inventory or equip my shop?"**), whereas regulatory statutes express norms through classical juristic terminology (**al-sigha**, **mahal al-aqd**, **tamwil**, **fasl**). Pure keyword matching fails across this register gap.
2. **Cross-Script & Cross-Lingual Asymmetry:** Authoritative legal texts are authored exclusively in formal Arabic script, while daily digital interactions in Tunisia arrive in **French**, **English**, **Modern Standard Arabic**, or **Franco-Arabic transliteration** (**"murabaha"**, **"ijara"**, **"sukuk"**). The system must bridge languages without distorting legal wording through naive machine translation.
3. **Zero Tolerance for Unsourced Numbers or Rules:** An invented profit rate, grace period, or statutory penalty threshold creates immediate legal exposure. Any production-candidate assistant must deterministically verify that **every single claim and every single digit** exists verbatim in an approved source before a token reaches the user's screen.

2.2 The North Star: Five Non-Negotiable Properties

Every architectural trade-off across both repositories was governed by five properties ordered strictly by priority:

Rank	Governing Property	Concrete Engineering Enforcement
1	**Grounded & Auditable**	Every claim paired with a contiguous ≥12-character verbatim quote; every digit verified against its cited source; deterministic refusal when evidence is insufficient.
2	**Compliant by Design**	Post-gate PII scrubbing (`EMAIL` , `PHONE` , Tunisian `RIB/IBAN` , `CIN`); masked customer slice projection; zero model training or retention on customer conversations.
3	**Metered & Network-Isolated**	Two-phase token/turn quota enforcement on public and authenticated channels; intelligence engine isolated on a private network unreachable from the Internet.
4	**Regulable by Bank Staff**	Staff back-office to author governed rates/facts (which outrank static documents), seal/rollback intent taxonomies, inspect chunk state, and audit every turn.
5	**First-Class Multilingual & RTL**	Native French, Arabic, and English interfaces with full right-to-left (`RTL`) layout geometry across landing, portal, chat, and taxonomy screens.

Table 2 — Priority-ordered North-Star properties governing all design choices.

2.3 What Was Delivered: Two Cooperating Systems, One Product

Rather than entangling banking user management, core-banking tables, and Python NLP experimentation inside a brittle monolith, the internship delivered **two cleanly decoupled systems** bound by a frozen, authenticated HTTP contract:

> **Architectural Boundary Invariant**
>
> **RAGLab** answers strictly from verified evidence; **Bank Assistant** serves customers, enforces identity and quotas, and governs what RAGLab knows. **Neither system ever writes into the other's data store.** Java opens the conversation turn in PostgreSQL **before** invoking RAGLab (making even downstream failures auditable to staff), while end-customers never see internal file paths.

- **Bank Assistant (Customer & Staff Web Application):** Provides a trilingual public portal, a quota-metered floating assistant widget, a full authenticated digital banking portal (`/portal` with accounts, masked cards, movements, Murabaha/Ijara simulations, and advisor booking), and an eleven-screen staff back-office (`/admin/**`) featuring a dedicated five-tab **RAGLab Console**.
- **RAGLab (Governed Intelligence Microservice):** Exposes 25+ versioned FastAPI routes implementing Arabic PDF restructuring, structure-aware chunking, dual 2048-d dense + BM25 indexing, rule-based intent and sufficiency reasoning, declarative legal knowledge units, three-stage citation gating, output PII scrubbing, and JSONL audit trails.

CHAPTER 3

System Architecture: The Full Picture

3.1 End-to-End Platform Topology

Figure 1 illustrates the complete runtime architecture connecting a user's browser to the governed intelligence engine across three isolated security zones.



Figure 1 — End-to-end platform architecture across Public DMZ, Bank Application Edge, and Private Intelligence Engine.

The end-to-end request path enforces strict single-responsibility boundaries:

1. **Browser to Nginx (`Zone 1`):** The client browser communicates exclusively over HTTPS with the Nginx container, which serves hashed Angular 21 static bundles (`index.html` strictly `no-cache`), injects security hardening headers (`HSTS`, `CSP`, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`), and proxies `/api/**` requests with buffering disabled so Server-Sent Events (SSE) stream smoothly.
2. **Nginx to Spring Boot 4 Edge (`Zone 2`):** The Java 21 backend authenticates JWTs, enforces role permissions and session token budgets, owns the **entire relational schema** in PostgreSQL 16 via Flyway migrations `V1..V15` (`spring.jpa.hibernate.ddl-auto=validate`), and records every user exchange prior to upstream invocation.
3. **Spring Boot Edge to RAGLab (`Zone 3`):** Over a private internal subnet, `AgentClient` invokes RAGLab via a constant-time verified `X-Service-Token` header and propagates a correlation `X-Turn-Id` header

across services. RAGLab maintains its own local ChromaDB cosine index and BM25 structures without ever touching PostgreSQL.

3.2 Deliberate Architectural Sobriety: Stack Justification

A conscious engineering decision runs throughout both repositories: **zero heavy orchestration frameworks (no LangChain, no LlamaIndex), zero external cloud vector databases, and zero opaque vendor SDK wrappers**. Every prompt, retry policy, chunk fingerprint, and gate rule can be read line by line during a compliance audit.

Architectural Layer	Selected Technology	Engineering Rationale & Rejected Alternatives
Frontend SPA	Angular 21 (standalone), TypeScript, Vitest, self-hosted Inter	Lazy route isolation for `/portal` and `/admin/**`; strict CSS budgets; self-hosted typography eliminates external Google Fonts runtime calls.
Application Edge	Java 21, Spring Boot 4, Flyway 15, JJWT 0.12, Virtual Threads	Strict `ddl-auto=validate` guarantees Flyway single ownership; JDK 21 virtual threads handle concurrent long-lived SSE streams without thread-pool starvation.
Intelligence Service	Python 3.11, FastAPI, Uvicorn, stdlib HTTP client	Explicit control flow makes sufficiency loops and citation gates 100% deterministic and inspectable; avoids framework version churn.
Vector & Lexical Store	ChromaDB local (`cosine`) + custom BM25 (`k1=1.5, b=0.75`)	Zero banking-document egress to third-party SaaS vector clouds; dual cryptographic fingerprints prevent serving mixed or stale embeddings.
Embeddings & LLM	NVIDIA `nemotron-3-embed-1b` (2048-d) + `qwen/qwen3.8-max`	Selected via paired benchmarks on real Arabic legal text; pinned explicitly with **zero silent fallback** so model degradation always fails loudly.
Tokenization	`tiktoken` (`cl100k_base`) + offline estimator	Deterministic token accounting encoded directly inside `chunk_fp v4` so mismatched tokenizers cannot pollute an index.
State & Queues	PostgreSQL 16 + Redis durable `BRPOP` queue	Transactional ACID guarantees for banking entities and audits; ordered, crash-safe background ingestion worker notifications.
Packaging & Ops	Docker Compose (5 services), `Dockerfile.prod` , SHA scripts	Identical immutable container digests promoted across `dev → staging → prod` , plus offline `.tar` bundle export and native Windows scripts.

Table 3 — Complete technology stack selection and governance rationale.

3.3 The Two Governed Operational Lifecycles

Every byte entering or leaving the platform follows one of two explicit state machines:

- **The Document Lifecycle:** Administrators upload or push normative files to RAGLab through `POST /documents`. Content is versioned by **SHA-256 byte hash**: uploading byte-identical content is an idempotent no-op, duplicating an existing corpus file triggers an explicit `409 document_already_in_corpus` guard, and updated content increments the document version cleanly. Indexing executes asynchronously via `POST /ingest` (strictly single-flight; concurrent calls receive an honest `409` busy state rather than corrupting the collection). RAGLab reports truth-on-disk index states (`pending` , `indexed` , `stale`) back to the admin console.
- **The Conversation Lifecycle:** Depicted step by step in Figure 2, every chat turn executes two-phase quota accounting, pre-call persistence, HTTP/1.1 streaming relay, citation verification, and post-gate PII redaction.

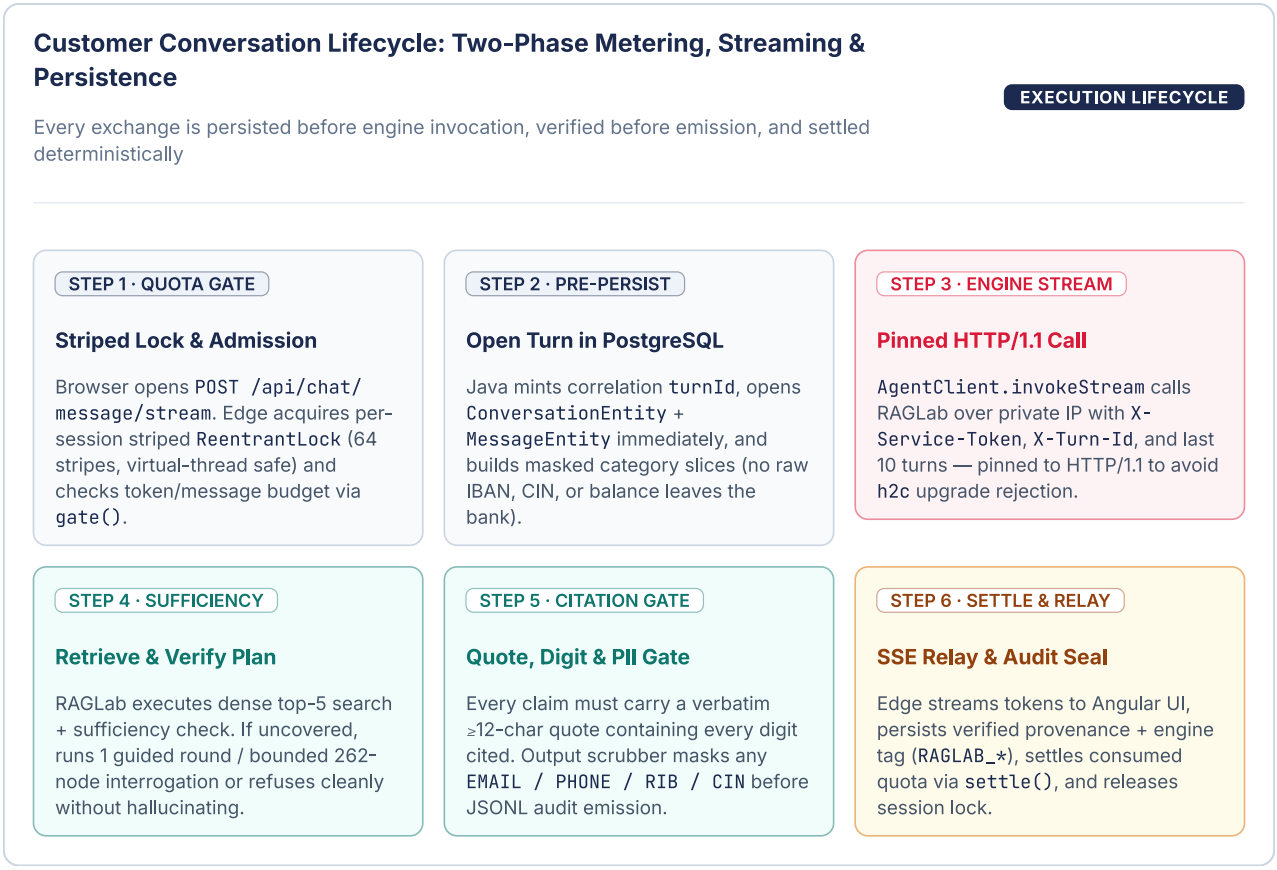


Figure 2 — Six-step customer chat execution path combining striped virtual-thread locks, pre-call persistence, and citation gating.

CHAPTER 4

The Intelligence Engine — RAGLab: How It Thinks

4.1 The Authoritative Arabic Corpus & Its Scanned-PDF Pathology

RAGLab is evaluated not on synthetic toy snippets, but on **four unabridged Arabic banking documents** used in real Tunisian Islamic banking practice:

Authoritative Document	Normative Role	Restructured Chunks
Law No. 2016-48 (198 articles, 11 titles)	Supreme statutory banking law	**233** fragments
BCT Circular No. 2019-08 (20 chapters)	Central Bank operational regulation	**13** fragments
Internal Islamic Operations Guide	Internal product & compliance procedures	**54** fragments
Madkhal ila al-Sayrafa al-Islamiya	Staff training primer & juristic definitions	**39** fragments
Total Governed Corpus	**4 real Arabic normative documents**	**339 fragments**

Table 4 — Composition of the governed Arabic corpus under structure-aware chunking.

Extracting text raw from Official Printing Office (**JORT**) and scanned PDFs initially yielded unusable noise: lines stored in **visual right-to-left presentation order** rather than logical reading order, reversed token sequences, corrupted numerals, and repeated page headers slicing articles apart. RAGLab solves this through an explicit dual-track architecture shown in Figure 3.

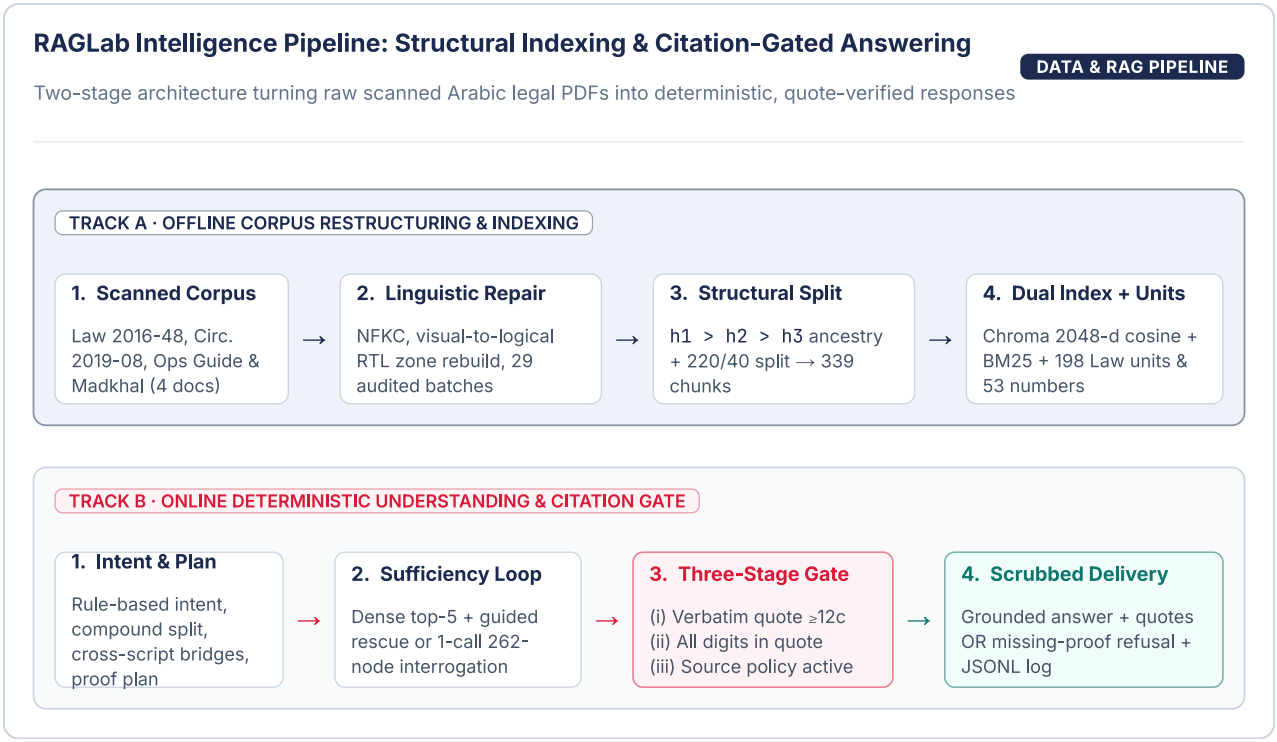


Figure 3 — RAGLab dual-track architecture: offline linguistic restructuring & indexing (Track A) and online deterministic gating (Track B).

4.2 Three-Stage Structure-Aware Chunking (`restructure` Mode)

To prove whether structural awareness genuinely improves retrieval, RAGLab retains both the baseline fixed-window splitter (`size` mode: 220 tokens, 40-token overlap) and the production `restructure` pipeline:

1. **Semantic & Linguistic Normalization:** Applies Unicode `NFKC` normalization, repairs Arabic presentation forms, unifies **alef/yah** variants, strips **tatweel** and vocalization noise, drops repeated running headers occurring ≥ 3 times, reconstructs visual-order zones into logical reading order, and extracts structural legal markers (**al-unwan**, **al-bab**, **al-qism**, **al-fasl**) into hierarchical Markdown (`#`, `##`, `###`).
2. **Ancestry Breadcrumb Enrichment:** Prepends the full hierarchical path (`Title > Chapter > Section > Article`) directly above every heading block and Markdown table so each fragment remains self-contextualizing when retrieved in isolation.
3. **Recursive Structural Splitting:** Splits preferentially along semantic boundaries (`["\n# ", "\n## ", "\n### ", "\n\n", "\n", " "]`) within the exact same `220 / 40` token budget. Every chunk embeds a verifiable metadata card carrying `document`, `chunk_index`, `heading`, `language`, `section_type`, `chunk_fp` (v4), `embedding_fp`, and `embedding_model`.

> **Measured Architectural Win — +20 Percentage Points**
>
> On the frozen 50-question benchmark using identical embeddings and budget, switching from fixed-window `size` to `restructure` jumped live `hit@1` from **51% to 71%** (+20 pp) and offline BM25 `hit@1` from **42% to 62%** — outperforming every downstream retrieval-fusion experiment.

4.3 High-Dimensional Embeddings, Dual Indexing & Fingerprint Guards

- **Embedding Pipeline:** Uses `nvidia/nemotron-3-embed-1b` at full native **2048 dimensions** (batch size 32, exponential backoff honoring `Retry-After`, recursive batch bisection if any vector returns degenerate norms, and deterministic disk caching).
- **Isolated Collection Namespace:** ChromaDB stores one persistent cosine collection per `(embedding_space × chunking_mode)` alongside a parallel BM25 index.

- **Fail-Closed Fingerprint Guard:** Before executing any query, `store.py` validates stored `chunk_fp` and `embedding_fp` against active runtime policy. Any mismatch aborts immediately before calling external APIs, preventing silent corruption across mixed embedding spaces.

4.4 Retrieval Modes: Why Pure Dense Retrieval Remains Default

Three candidate retrieval strategies were benchmarked side by side under identical conditions on the frozen 50-question evaluation suite (`top-20` candidate pool narrowed to `top-5` evidence context):

Retrieval Strategy	hit@1 (%)	hit@3 (%)	hit@5 (%)
`vector` — **Pure Dense Cosine (Default)**	**71%**	**80%**	**87%**
`rrf` — Reciprocal Rank Fusion (`k = 60`)	71%	80%	82%
`blend` — Linear Score Fusion (`λ = 0.7`)	62%	80%	84%

Table 5 — Live paired retrieval evaluation across all 50 benchmark questions.

On cross-lingual French questions asking about Arabic legal articles, the advantage of pure dense retrieval is even sharper: `50 / 67 / 67` for dense versus `33 / 33 / 50` for RRF, because lexical BM25 scores zero cross-script overlap and drags down valid semantic matches. Consequently, **pure dense vector search remains the production default**, enhanced by a deterministic feature-based second-pass reranker proven to lift offline `hit@3/@5` from `62/82/87` to `62/89/89`.

4.5 Deterministic Question Understanding & Bounded Interrogation

Before invoking a generative LLM, RAGLab runs an explicit, deterministic reasoning pipeline:

Understanding Module	Deterministic Responsibility & Safety Bound
`intent.py`	Rule-ordered intent classifier (first match wins); splits compound multi-part questions and flags influential ambiguity explicitly.
`evidence_plan.py`	Constructs a declarative **proof plan** specifying what evidentiary slots must be covered — never pre-assuming the answer.
`sufficiency.py`	Evaluates retrieved chunks against the proof plan, emitting `sufficient`, `insufficient`, `contradictory`, or `indeterminate`, and triggering one bounded guided rescue search when needed.
`interrogate.py` + `topic_map.py`	**Executed only when coverage is zero:** makes at most **one** bounded LLM call mapping colloquial wording onto a closed **262-node topic catalog** (198 law articles + 20 circular chapters + corpus sections). Any invented topic, malformed schema, or circular paraphrase fails closed into an honest refusal.

Table 6 — Modules composing RAGLab's pre-generation understanding layer.

Whenever interrogation re-aims a colloquial question, full provenance is disclosed in the API response (`original_question`, `understood_as`, `scope_note`, and exact `interrogation` latency) — never hidden behind silent query rewriting.

4.6 Delivery Layer: Three-Stage Citation Gate, Scrubbing & Declarative Law Units

Generation operates under prompt `grounded-v1`, which frames both user questions and retrieved passages as **untrusted data**, disables tool calling, and enforces strict JSON output `{answerable, claims: [{text, evidence: [{source_id, quote}]}]}`. Every claim must pass **three deterministic post-generation gates**:

1. **Verbatim Quote Membership Gate:** After Unicode `NFKC`, casefolding, and whitespace normalization, every cited `quote` ($\geq$ 12 characters) must be an exact contiguous substring of the referenced source chunk.
2. **Strict Numeric Fidelity Gate:** Every digit sequence appearing in a claim (supporting decimals, thousand separators, and both Western and Arabic-Indic numerals) must appear literally inside its attached quotes. Any synthesized, rounded, or calculated figure triggers an immediate `unsourced_number` refusal.

3. **Governance Policy Gate:** Every cited chunk must belong to the active retrieval window, satisfy allowed document scope, and carry `nafidh` (**in force**) status on the 6-axis governance metadata.

Only **after** passing all three gates (preserving exact digit checks) does `scrub.py` redact personal identifiers (`EMAIL`, `PHONE`, 20-digit Tunisian `RIB`, `TN59` `IBAN`, contextual `CIN`) into `[LABEL]` tokens and append a structured entry to `GET /audit`. Finally, on Law No. 2016-48, a curated **Declarative Knowledge Layer** encodes all **198 articles** (`loi-2016-48:artNNN`), **77 internal cross-references**, and **53 structured legal numbers** (`GET /numbers`), raising law retrieval accuracy from `67/92/92` to a perfect `100% / 100% / 100%`.

CHAPTER 5

The Digital Bank — Bank Assistant Web Application

5.1 Three Segregated Channels & Identity Isolation

Bank Assistant is the sole public-facing edge of the platform. Rather than blending anonymous visitors, retail banking clients, and compliance staff inside shared components, the Angular 21 and Spring Boot 4 application enforces **three strictly segregated functional surfaces**, detailed architecturally in Figure 4.

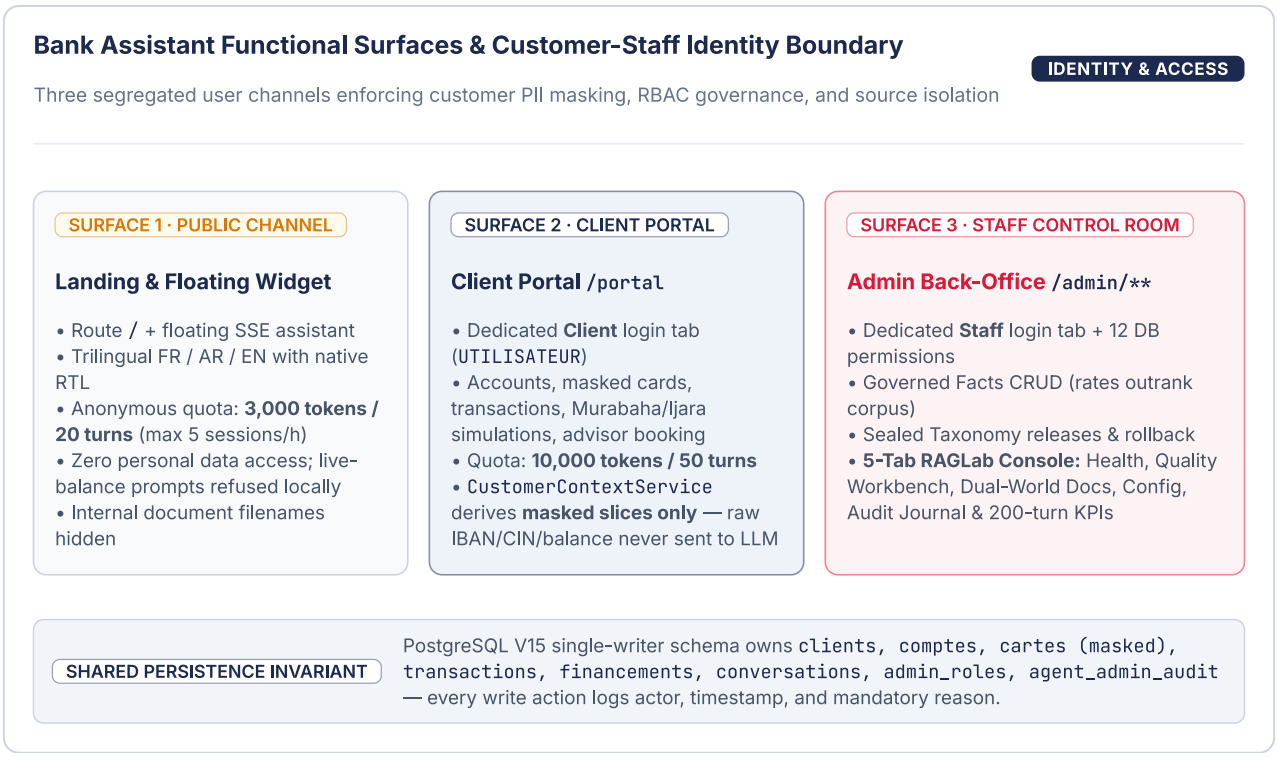


Figure 4 — Functional architecture of Bank Assistant illustrating strict separation across Public, Client Portal, and Staff Governance surfaces.

- **Public Experience (`/` & Floating Widget):** Delivers a trilingual (FR / AR / EN) institutional landing showcase styled with official brand tokens (`#E0122F` Red, `#F5881C` Orange) and symmetric RTL layout mirroring. Visitors can query the floating SSE assistant immediately without login under strict anonymous session budgets (**3,000 tokens / 20 messages**, max 5 sessions/hour).
- **Split Authentication & Client Portal (`/portal`):** Login explicitly separates **Client** and **Staff** authentication flows. Authenticated retail clients (`UTILISATEUR`) access real-time account balances, masked cards, transaction histories, Sharia financing simulations (**Murabaha** / **Ijara**), advisor appointment scheduling, and a persistent multi-conversation assistant carrying their last 10 turns.

- **Customer Context Masking (`CustomerContextService`):** A pivotal privacy design ensures raw balances, IBANs, card numbers, or CINs **never** leave PostgreSQL toward the LLM. When a signed-in client asks a personalized question, Java derives coarse non-identifying category slices in memory at call time.

5.2 Core Banking Relational Domain (`Flyway V1 .. V15`)

Schema V15 models realistic retail and participative core banking cleanly under single Flyway ownership:

Banking Domain	Owned PostgreSQL Tables & Privacy-Critical Design
Identity & KYC	`clients` (`CIN` , internal ref `ABK######` , segment, `langue_preferee` , KYC status, risk tier, `agence_id` , employer, income bracket).
Accounts & Masked Cards	`comptes` (`IBAN` , `RIB` , `solde` , `solde_disponible` , status), `cartes` (stores masked `4571 **** **** 6842` only — **full PAN and CVV never stored**), `transactions` (dates, counterparty, channel, amount).
Islamic Financing	`financements` (contract reference, structure **Murabaha / Ijara**, asset purpose, principal, disclosed profit margin, schedule, `capital_restant`), `beneficiaires` .
Advisory & Conversations	`advisor_requests` (motive, branch, preferred slot, free-text note), `conversations` , `public_chat_session` , `messages` (scrubbed Q&A, `sources_utilisees` , `trace_json`).

Table 7 — Core banking relational schema owned exclusively by Spring Boot + Flyway.

5.3 Token-Family Security, Dynamic RBAC & Virtual-Thread Metering

Three backend mechanisms harden daily banking operations:

1. **JWT Rotation with Theft Detection:** Short-lived access tokens ($\leq$ 1 hour) pair with single-use refresh tokens grouped by cryptographic `family` . Presenting an already-rotated `jti` immediately invalidates the **entire token family** (`401`), neutralizing replay attacks. Any role mutation or account deactivation purges active sessions instantly.
2. **Database-Backed Dynamic RBAC (`V14`):** Staff roles compose **12 fixed server-enforced permissions** across `admin_roles` , `admin_permissions` , and `admin_access_audit` . `SUPER_ADMIN` cannot be orphaned, unknown permissions fail closed (`403`), and every permission change requires a written audit motive.
3. **Striped Locks for JDK 21 Virtual Threads:** Two-phase metering calls `gate()` before RAGLab invocation and `settle()` upon stream completion. Using `synchronized` blocks around streaming calls would pin JVM carrier threads; instead, a **64-stripe** `ReentrantLock` **table** serializes per-session quota checks cleanly without carrier pinning.

5.4 The Staff Control Room & Five-Tab RAGLab Console

Admin Route	Operational Governance Capability
`/admin/facts`	**Governed Facts CRUD:** Author authoritative bank rates, fees, and eligibility rules with append-only audit (`business_facts_audit`) and mandatory reason — facts deterministically outrank corpus text.
`/admin/taxonomy`	**Intent Taxonomy Releases:** Edit multilingual exemplars, `seal()` immutable numbered releases (`CatalogRelease`), and roll back cleanly in one click.
`/admin/agent/health`	**Engine Connectivity:** Inspect active `RAGLAB_URL` (hot-swappable by `SUPER_ADMIN` with audit log) and live `/health` latency probes.
`/admin/agent/quality`	**Quality Workbench:** Interactive verification testing whether answers carry quotes, whether retrieval found expected passages, and whether quotes exist verbatim.
`/admin/agent/documents`	**Dual-World Corpus Control:** Unified upload drop zone displaying honest per-document states (`pending` , `indexed` , `stale`) across both storage worlds and triggering batch `/ingest` .
`/admin/agent/config` & `journal`	**Model Config & Audit Journal:** Write-only key management with fingerprint display, plus full searchable history of every administrative read/write and motive.
`/admin/agent/monitor`	**Live Turn Telemetry:** Real-time stream of exchanges computing grounded rate, source coverage, and refusal breakdown across the latest 200 turns.

Table 8 — Administrative governance screens under /admin/** (StaffGuard protected).

CHAPTER 6

Engineering Challenges Solved: What Broke & How It Was Fixed

The true measure of an engineering project lies not in its initial diagram, but in how real-world defects were diagnosed, measured, and permanently eliminated through architecture. Nine high-impact production problems were resolved during this work.

6.1 Linguistic & Retrieval Breakthroughs

1. **Visual-Order Scanned Arabic PDFs & Corrupted Numerals:** Raw extraction from Official Gazette PDFs inverted Arabic words into visual presentation order and garbled article numbers. **Fix:** Built `restructure.py` combining NFKC normalization, repeated-header stripping, legal marker parsing, and zone-level RTL reconstruction, complemented by **29 supervised repair batches** on Law 2016-48 (215 verified corrections) plus 35/23/21 batches across the remaining three texts — all regression-checked via `llm_repair_check.py` while keeping raw source files in `docs/` strictly immutable.
2. **Fixed 220-Token Windows Severing Legal Definitions:** Sliding windows cut statutory articles mid-sentence, separating conditions from exceptions. **Fix:** Three-step structure-aware chunking with `h1 > h2 > h3` breadcrumbs, yielding our single largest jump: **+20 pp** `hit@1` (`51% → 71%` live; `42% → 62%` offline), visualized in Figure 5.
3. **Cross-Script Gap (`"murabaha"` vs Arabic Script):** Latin-script banking terms have zero lexical intersection with Arabic passages. **Fix:** Combined 2048-d multilingual embeddings, document-frequency-guarded cross-script bridges validated by `diag_bridge.py` (preventing high-frequency boilerplate from anchoring false matches), curated colloquial concept mappings (**bar/casino → khamr/maysir** with explicit `scope_note` disclosure), and bounded 262-node topic interrogation.
4. **Silent Duplicate Chunk Inflation (859 / 1,713 Chunks):** Re-uploading documents previously appended duplicate vectors, skewing BM25 term frequencies and filling top-5 slots with identical text. **Fix:** Enforced SHA-256 content-hash `409` deduplication on `POST /documents` , single-flight `POST /ingest` , and strict JSONB source-purge verification before upsert.

5. **French Substring Regex Misclassification (`"Quelle"` Inside `"Quelles"`):** Unanchored French intent patterns matched `Quelle` inside plural procedural questions (`Quelles sont les démarches...`), routing them as simple definitions. **Fix:** Word-boundary anchored regexes backed by live-corpus regression tests.



Figure 5 — Comparative hit@1 / hit@3 / hit@5 progression across chunking, fusion, reranking, and declarative unit indexing.

6.2 Distributed Systems & Production Runtime Fixes

6. **Guided Rescue Round Silently Inert in Production:** Unit tests passed a mock search callable to `sufficiency.py`, whereas the live FastAPI route omitted it — leaving second-round rescue retrieval completely dead in production. **Fix:** Wired the live search callable into service initialization and established a project-wide rule: **every capability claim must be verified against the running HTTP container**.
7. **Java HTTP/2 `h2c` Upgrade Rejected With `422` by Uvicorn:** `java.net.http.HttpClient` defaults to HTTP/2 cleartext (`Upgrade: h2c`) over internal HTTP, causing Uvicorn to reject streaming requests with `422 Unprocessable Entity`. **Fix:** Explicitly pinned `AgentClient.invokeStream` to `HttpClient.Version.HTTP_1_1` — a one-line root fix after deep protocol tracing.
8. **Docker Compose Secret Precedence & Upstream Error Masking:** A default Compose token overrode operator environment variables, while upstream LLM timeouts surfaced as opaque `500` errors. **Fix:** Inverted variable precedence so environment secrets always win, and mapped upstream failures cleanly to `502 Bad Gateway` with scrubbed diagnostics.

9. **Nginx Stale `index.html` Caching & CSP `onload` Blocking:** Browser caching of `index.html` served stale bundle hashes after redeployments, while Angular critical-CSS inlining injected inline `onload` scripts blocked by our strict Content Security Policy. **Fix:** Enforced `Cache-Control: no-store, no-cache` on `index.html` and disabled inline critical-CSS rewriting.

CHAPTER 7

Security, Privacy & Data Governance

7.1 Seven-Layer Defense-in-Depth Architecture

Banking software cannot rely on perimeter security alone. As illustrated in Figure 6, seven independent defensive layers reinforce one another so a single misconfiguration cannot expose customer records or produce unverified banking claims.

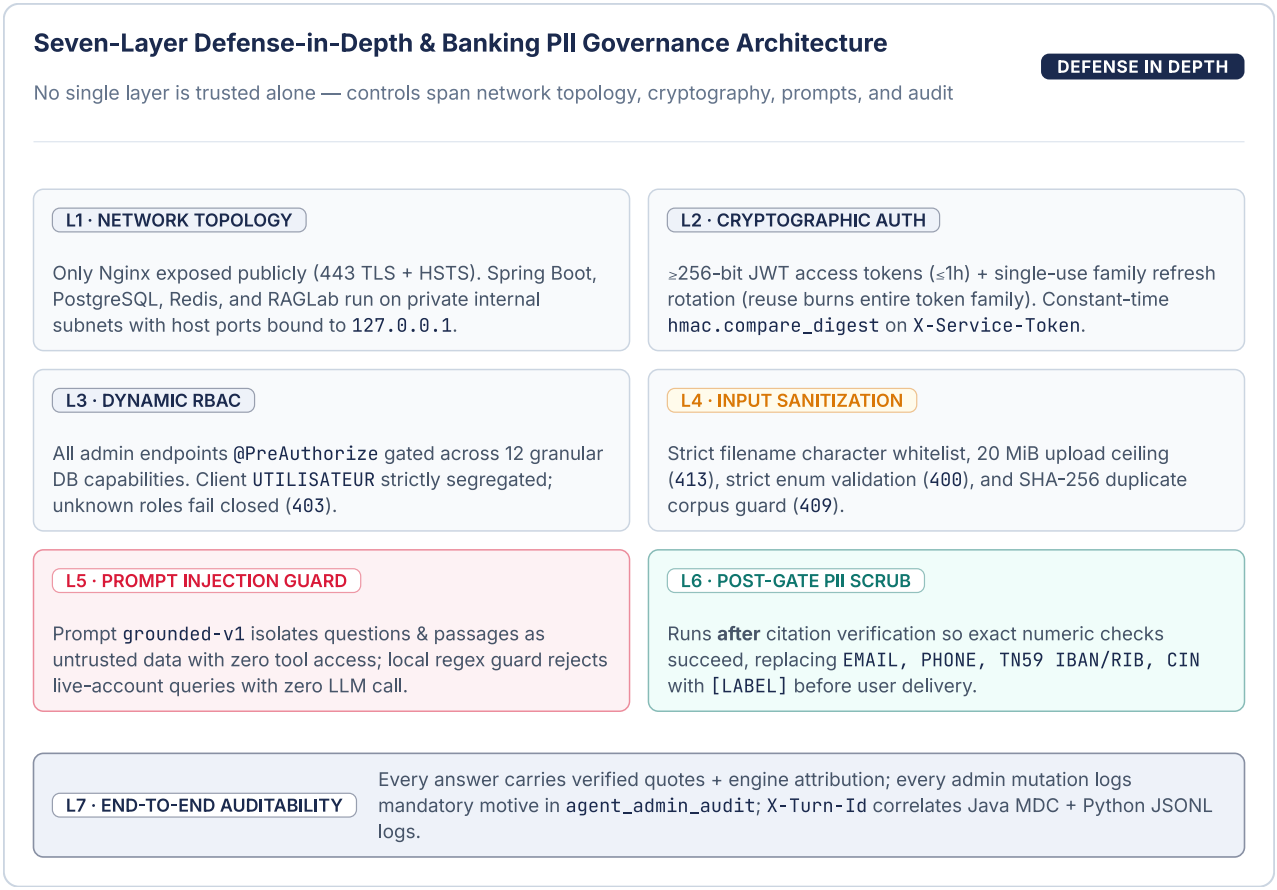


Figure 6 — Seven-layer defense-in-depth model spanning network isolation, cryptography, RBAC, prompt isolation, PII scrubbing, and audit.

7.2 Personal Data Inventory, Masking & Honest Gap Disclosure

Across nine relational table groups (`clients`, `comptes`, `cartes`, `transactions`, `beneficiaires`, `financements`, `advisor_requests`, `conversations`, `refresh_tokens`), two architectural guarantees and two documented pre-production obligations govern personal data:

- **Zero Raw PII Egress to Intelligence Models:** Neither customer identifiers (`CIN`, `ABK` ref, email) nor financial numbers (`IBAN`, `RIB`, balances) are ever sent to RAGLab or external model endpoints; only coarse category slices generated by `CustomerContextService` cross the boundary when authenticated context is required.

- **Card Data Minimization & Third-Party Awareness:** Full card numbers (`PAN`) and `CVV` codes are never persisted anywhere in the database (`cartes.porteur` stores pre-masked display strings). Counterparty strings in `transactions` are documented as potential third-party personal data for Data Protection Impact Assessment (DPIA) review.
- **Free-Text Chat Risk & Documented Pre-Production Gaps:** Because customers may type an IBAN or national ID voluntarily inside a chat message (`messages.question`), output scrubbing and audit logs mitigate leakage, while automated **input-side PII redaction** and **statutory data retention/deletion schedules** are explicitly documented for legal sign-off prior to unsupervised production launch.

7.3 Threat Pass (`THREAT_PASS_6.2`) & End-to-End Traceability

A systematic 20-point security review drove concrete hardening across both codebases: eliminating a legacy unauthenticated endpoint, enforcing constant-time `hmac.compare_digest` token checks, disabling Swagger UI under `prod` , binding local container ports strictly to `127.0.0.1` , and correlating every turn across Java Log4j `MDC` and Python `contextvars` via `X-Turn-Id` . Open items remain transparently tracked rather than concealed: binary magic-byte upload inspection, Redis authentication in HA clusters, and scheduled purging of refresh tokens expired beyond 30 days.

CHAPTER 8

Deployment, Promotion & Operations

8.1 Build Once, Promote Everywhere (`dev → staging → prod`)

Operational reliability rests on a strict rule enforced by `deploy/promote.sh` : **container images are built once from a clean Git commit SHA, verified, and promoted by immutable digest — never rebuilt for production**.

Dimension	Development (dev)	Staging (staging)	Production (prod)
Orchestration	`docker compose up`	Same image SHA + real TLS	Same image SHA + HA stores
Spring Profile	`default`	`prod`	`prod`
Swagger / OpenAPI	Enabled	Disabled (`404`)	Disabled (`404`)
Log Format	Human / JSON	Structured `ECS` JSON	Structured `ECS` JSON
Secret Injection	Local `.env` (gitignored)	Platform secret mounts	Platform secret mounts
Database State	Seeded synthetic fixtures	Monthly anonymized snap-shot	Live customer data + PITR

Table 9 — Environment parity matrix across development, staging, and production.

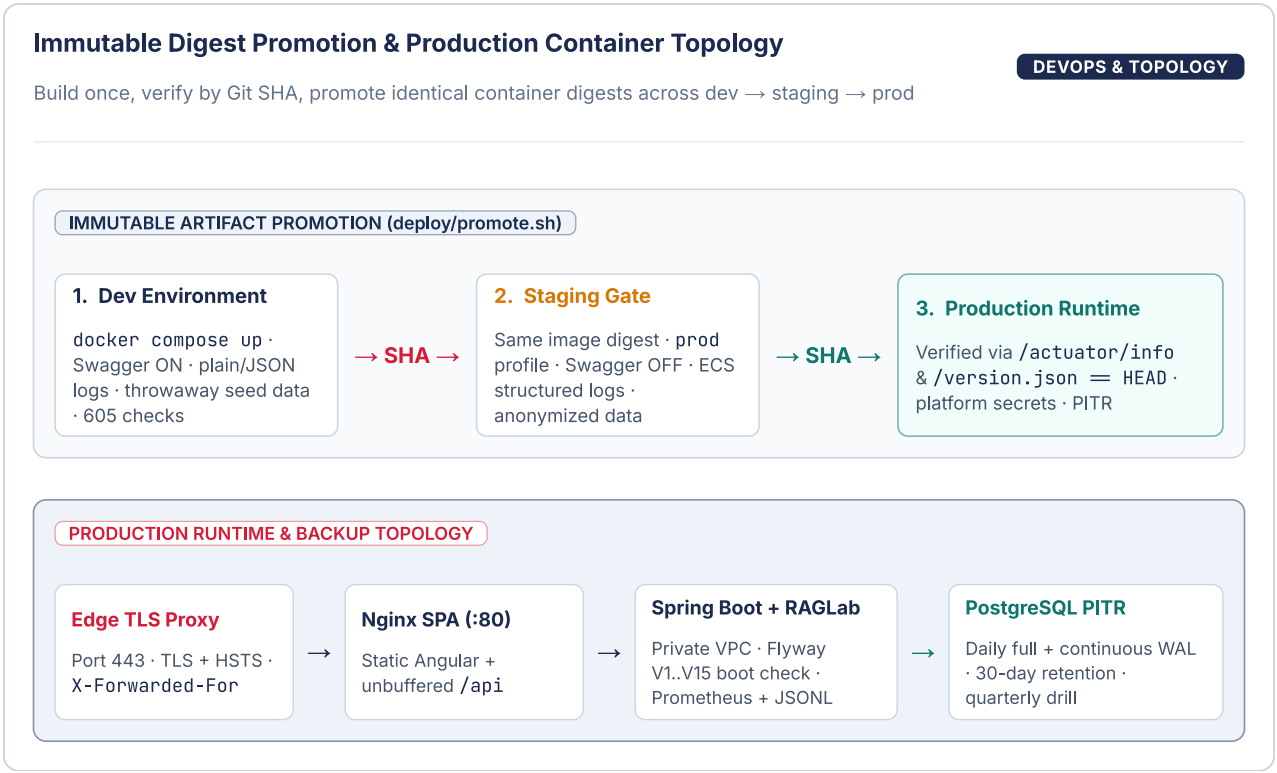


Figure 7 — Immutable Git-SHA container promotion workflow and production runtime network/backup topology.

8.2 Zero-Surprise Rollout & Observability Runbook

Database evolution follows strict **expand → migrate → contract** discipline across Flyway migrations `V1..V15` (including refactoring a legacy non-idempotent `V1` baseline so clean CI verification runs on every commit). Production rollouts execute deterministically:

```
TAG=v1.5.0 docker compose -f docker-compose.yml -f docker-compose.prod.yml pull
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d backend
docker compose logs backend | grep -E "Flyway|Migrating|Successfully"
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d frontend
E2E_BASE_URL=https://assistant.example.tn ./e2e/run.sh
```

Prometheus endpoints (`GET /actuator/prometheus` on Java exposing `chat_turns_total` and latency histograms; `GET /metrics` on RAGLab exposing `agent_turns_total` and cache counters) pair with `RUNBOOK.md` and `HOST_TESTING.md` so any operator can spin up a seeded workstation stack (staff across all roles plus four synthetic clients) in a single command.

CHAPTER 9

Evaluation Results, Benchmarks & Limitations

9.1 Frozen Evaluation Protocol

All quantitative claims obey strict scientific hygiene across two frozen datasets that are **never edited to chase higher scores**:

- `questions_50.json` **(50 Main Cases):** 10 verbatim lookups, 20 lexical paraphrases, 15 cross-lingual queries (6 French + 9 English), and 5 out-of-scope traps across 32 Arabic, 8 French, and 10 English prompts — each anchored to a verified distinctive substring.
- `questions_targets.json` **(10 Diagnostic Arabic Cases):** 2 cases across each of 5 linguistic stress categories **(colloquialisms, synonyms, implicit intent, compound multi-part requests, and influential ambi-**

guity). Paired arms always run inside the same session within a documented ±2 pp live-serving variance band.

9.2 Consolidated Benchmark & Robustness Scorecard

Evaluation Dimension	Measured Outcome on Frozen Artifacts
Chunking Architecture (`50` live)	`size` `51 / 64 / 71` → `restructure` `71 / 80 / 87` (`hit@1/3/5`) — **+20 pp** `hit@1` breakthrough.
Retrieval Strategy (`50` live)	**Dense** `71 / 80 / 87` outperforms `RRF` (`71 / 80 / 82`) and `blend` (`62 / 80 / 84`); French dense `50/67/67` vs `33/33/50` .
Second-Pass Reranker (offline)	`62 / 82 / 87` → `62 / 89 / 89` (+7 pp `hit@3`), activated cleanly.
Law 2016-48 Unit Index (`12` cases)	`67 / 92 / 92` baseline → `100% / 100% / 100%` — every legal article retrieved at **rank 1**.
Stress Categories (`10` AR cases)	Compound `100%` · Implicit & Ambiguous `50% / 100%` · Synonyms `50%` · Raw colloquial `0%` (mitigated via curated bridge lexicon).
Cross-Model Independence	Decision agreement `0.720` ; cited-source Jaccard similarity `1.000` whenever both models answer (zero source disagreement).
Adversarial Gate Suite (`6` attacks)	Fabricated quote, borrowed cross-chunk quote, calculated unsourced number, unknown ID, inline marker, and prompt injection → **6 / 6 deterministically rejected**.
Automated CI Verification	**472** unit tests + **133** offline checks (**605 total**), **8** GitHub Actions workflows, **469** hard-harness families — all green.

Table 10 — Master scorecard of measured retrieval, grounding, robustness, and test metrics.

9.3 Explicit Engineering Limitations & Readiness Verdict

Intellectual honesty is part of banking safety. Six concrete limitations remain explicitly documented:

1. **Lexical Membership ≠ Logical Entailment:** The citation gate proves that quoted words and digits exist verbatim in the source, not that a claim preserves every conditional nuance (e.g., negations outside the quote window).
2. **No Hard Cosine Cutoff:** Top-5 chunks are passed to sufficiency evaluation even on out-of-domain queries; safety relies on the sufficiency controller and citation gates rather than an arbitrary similarity threshold.
3. **Single-Worker Ingestion & Index Footprint:** RAGLab processes one ingestion job at a time per instance; horizontal multi-replica scaling is architected on paper rather than deployed.
4. **Batch Re-Ingestion Updates:** Edited documents remain marked `stale` until batch `POST /ingest` completes.
5. **Evaluation Scale:** While rigorous, `50 + 10` benchmark prompts plus `469` harness families constitute an engineering validation suite, not a formal statistical guarantee over all possible customer phrasing.

Official Readiness Verdict

Ready for supervised internal pilot deployment on approved non-confidential normative banking documents; **not yet certified for unsupervised direct-to-customer financial advice** pending legal sign-off on retention schedules and entailment review. Documenting this boundary clearly is itself a core governance deliverable.

CHAPTER 10

Conclusion & Perspectives

10.1 Synthesis of Contributions

This internship demonstrated that generative AI can be integrated into an Islamic banking environment **without surrendering determinism, regulatory compliance, or architectural clarity**. Across two production-structured codebases, we delivered:

- **A four-layer governed RAG engine (`RAGLab`)** transforming noisy scanned Arabic legal PDFs into 339 hierarchy-enriched fragments (+20 pp `hit@1`), paired with a 198-article declarative Law 2016-48 index (`100%` rank-1 recall) and a three-stage verbatim/numeric citation gate.
- **A full-stack trilingual digital banking application (`Bank Assistant`)** enforcing client/staff isolation, token-family rotation, virtual-thread-safe quota metering, masked customer slices, and a five-tab staff governance workbench.

10.2 Three Engineering Lessons That Outlive the Code

1. **Paired Measurement Replaces Intuition:** Several plausible ideas — score blending, unguarded lexical expansion, raw window splitting — degraded cross-lingual accuracy and were rejected cleanly only because frozen paired benchmarks measured every change.
2. **Data Hygiene Beats Model Complexity:** Investing engineering effort into reconstructing scanned Arabic legal structure (`restructure.py`) delivered ten times the accuracy gain of tuning retrieval fusion parameters.
3. **Production Reality Is the Ultimate Judge:** Subtle integration bugs — HTTP/2 `h2c` streaming rejections, unpassed production callables, CSP script blocking — never appeared in unit mocks and were caught solely through end-to-end container testing.

10.3 Perspectives & Roadmap Toward Production

Five concrete engineering tracks extend this foundation toward full production scale: (i) per-document RBAC scoping and hierarchical chunk metadata (`path / level / parent_id`); (ii) natural-language entailment verification atop verbatim quote checks; (iii) OpenTelemetry distributed tracing complementing `X-Turn-Id` logs; (iv) magic-byte upload inspection plus automated input-side PII redaction; and (v) legal-approved conversation retention and purge jobs.

> *"The most trustworthy banking intelligence is not the system that answers the highest fraction of questions, but the system that knows precisely when not to answer — and can prove every word when it does."*
>
> Farah AYARI · ESPRIT × Al Baraka Bank Tunisia · Supervised by Mrs. Ahlem Ben Haddoud

References & Appendices

Primary Repository & Regulatory References

Reference Artifact / Source	Scope & Verified Contents
`RAGLab/README.md` `RAGLab/RAGLAB_SPEC.md`	Canonical pipeline specification, paired A/B benchmark tables, observed invariants.
`RAGLab/RAGLAB_GAP_ANALYSIS.md` `RAGLab/RAGLAB_ROADMAP.md`	Step-and-gate execution protocol, rejected hypotheses, and activation criteria.
`RAGLab/raglab/CONTRACT.md` `RAGLab/raglab/READINESS.md`	Frozen HTTP API contract (v1.5.0), error envelopes, and pilot readiness verdict.
`RAGLab/raglab/audits/MASTER_INDEX.md`	29-batch Arabic legal repair logs, SHA-256 codices, and quote verification scripts.
`bank-assistant/README.md` `bank-assistant/PLAN_TO_PROD.md`	Edge architecture, Flyway `V1..V15` ownership, and phased delivery milestones.
`bank-assistant/docs/CUSTOMER_PII_REPORT.md`	Field-by-field PII inventory, `CustomerContextService` masking, and DPIA notes.
`bank-assistant/docs/THREAT_PASS_6.2.md`	20-point security hardening audit, resolved vectors, and tracked pre-prod items.
Tunisian Law No. 2016-48 (**JORT** No. 58, 2016)	198 statutory articles governing banks and Islamic financial operations (Arts. 12–16).
Central Bank Circular BCT No. 2019-08	20 regulatory chapters framing participative banking operations in Tunisia.
Al Baraka Bank Tunisia & ESPRIT Institutional Docs	Official history (1983–present, 44 branches, IFN Awards 2024) and CTI EUR-ACE accreditation.

Table 11 — Authoritative internal engineering artifacts and statutory references.

Appendix A — Bilingual Technical & Islamic Banking Glossary

Term / Concept	Technical or Juristic Definition in Context
Al-Murabaha — المرابحة	Cost-plus asset sale in which acquisition cost and profit margin are explicitly disclosed and agreed upfront.
Al-Ijara — الإجارة	Sharia-compliant lease contract conferring usufruct of an asset, optionally paired with transfer of ownership.
Al-Sigha — الصيغة	Contractual form governing valid offer and acceptance under Islamic jurisprudence.
Al-Fasl — الفصل	Statutory article in parliamentary laws or numbered chapter within BCT circulars.
Nafidh — نافذ	Governance metadata state confirming a normative text is legally active and in force.
Kafin / Ghayr Kafin	Sufficiency verdicts emitted by `sufficiency.py`: **Sufficient** (كافٍ), **Insufficient** (غير كافٍ), or **Contradictory** (متعارض).
`hit@k` / `RRF`	Fraction of queries whose expected passage appears in the top-k results; **Reciprocal Rank Fusion** across rankers.
Three-Stage Citation Gate	Deterministic post-LLM filter enforcing verbatim substring membership, exact digit presence, and source policy validity.

Table 12 — Key domain and engineering terminology used across the report.

Appendix B — Consolidated Quantitative Fact Sheet

Metric Domain	Verified Repository Figures
RAGLab Codebase	376 commits · 249 tracked files · 86 Python modules (~40.8k lines) · 68 Markdown specs · Service `v1.5.0`.
Normative Corpus	4 real Arabic documents → 339 restructured chunks (`233` Law + `54` Guide + `39` Madkhal + `13` Circular) + `198` Law units.
Bank Assistant Codebase	255 commits · 428 files · 164 Java files (14,637 lines) · 87 TypeScript files (9,369 lines) · 15 Flyway migrations · 18 screens.
Verification & CI	472 Python unit tests + 133 offline checks + 123 Java backend tests + 91 Angular Vitest specs · 8 CI workflows.

Table 13 — Summary quantitative audit of both internship repositories.

Appendix C — Local Verification & Reproduction Guide

```
# 1. Governed Intelligence Engine (RAGLab)
cd RAGLab/raglab && python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt && ./run_tests.sh --offline
python main.py ingest --reset --data-dir ../docs
python main.py answer "Quelles sont les conditions de la Mourabaha ?" --query-lang fr

# 2. Full-Stack Bank Assistant Platform (Docker Compose)
cp bank-assistant/.env.docker.example .env   # configure RAGLAB_URL, RAGLAB_TOKEN, JWT_SECRET
docker compose up --build -d                 # serves SPA + API at http://localhost:8080

# 3. End-to-End Health Verification
curl -fsS http://localhost:8000/health
curl -fsS http://localhost:8080/api/agent/health -H "Authorization: Bearer <staff_jwt>"
```

Internship Report — Final Verification & Sign-Off Statement

Prepared by **Farah AYARI** (Software Engineering Department, ESPRIT) · Hosted by **Al Baraka Bank Tunisia** (Centre Urbain Nord, Tunis) · Supervised by **Mrs. Ahlem Ben Haddoud**. All metrics trace to frozen repository benchmarks; zero customer secrets or private URLs are included.

2025 – 2026